

```
# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

plt.style.use('ggplot')

users = pd.read_csv("users.csv")
photos = pd.read_csv("photos.csv")
likes = pd.read_csv("likes.csv")
comments = pd.read_csv("comments.csv")
tags = pd.read_csv("tags.csv")
photo_tags = pd.read_csv("photo_tags.csv")
follows = pd.read_csv("follows.csv")

print("All datasets loaded successfully!")
```

All datasets loaded successfully!

```
print("Users:", users.shape)
print("Photos:", photos.shape)
print("Likes:", likes.shape)
print("Comments:", comments.shape)
print("Tags:", tags.shape)
print("Photo Tags:", photo_tags.shape)
print("Follows:", follows.shape)
```

```
Users: (100, 6)
Photos: (257, 6)
Likes: (8782, 5)
Comments: (7488, 8)
Tags: (21, 4)
Photo Tags: (501, 3)
Follows: (7623, 5)
```

```
datasets = {
    "Users": users,
    "Photos": photos,
    "Likes": likes,
    "Comments": comments,
    "Tags": tags,
    "Photo Tags": photo_tags,
    "Follows": follows
}
```

```
for name, df in datasets.items():
    print(f"\n{name}")
    print(df.isnull().sum())
```

Users

id	0
name	0
created time	0
private/public	0
post count	0
Verified status	0

dtype: int64

Photos

id	0
image link	0
user ID	0
created dat	0
Insta filter used	0
photo type	0

dtype: int64

Likes

user	0
photo	0
created time	0
following or not	0
like type	0

dtype: int64

Comments

id	0
comment	0
User id	0
Photo id	0
created Timestamp	0
posted date	0
emoji used	0
Hashtags used count	0

dtype: int64

Tags

id	0
tag text	0
created time	0
location	0

dtype: int64

Photo Tags

photo	0
tag ID	0
user id	0

dtype: int64

Follows

```
follower          0
followee          0
created time      0
is follower active 0
followee Acc status 0
dtype: int64
```

```
# Convert date columns
```

```
users["created time"] = pd.to_datetime(users["created time"])
photos["created dat"] = pd.to_datetime(photos["created dat"])
```

```
# Extract useful features
```

```
photos["Posting_Hour"] = photos["created dat"].dt.hour
photos["Posting_Day"] = photos["created dat"].dt.day_name()
photos["Posting_Month"] = photos["created dat"].dt.month_name()
```

```
photos.head()
```

```
/tmp/ipykernel_1226/4138288954.py:2: UserWarning: Parsing dates in %d-
%m-%Y %H:%M format when dayfirst=False (the default) was specified.
Pass `dayfirst=True` or specify a format to silence this warning.
```

```
users["created time"] = pd.to_datetime(users["created time"])
/tmp/ipykernel_1226/4138288954.py:3: UserWarning: Parsing dates in %d-
%m-%Y %H:%M format when dayfirst=False (the default) was specified.
Pass `dayfirst=True` or specify a format to silence this warning.
photos["created dat"] = pd.to_datetime(photos["created dat"])
```

```
{"summary":{"name": "photos", "rows": 257,
"fields": [{"column": "id", "properties":
{"dtype": "number", "std": 74,
"min": 1, "max": 257, "num_unique_values":
257, "samples": [166, 187, 83],
"semantic_type": "",
"description": ""}],
"column":
"image link", "properties": {"dtype":
"string", "num_unique_values": 257, "samples":
["http://kaleigh.biz",
"http://germaine.name",
"http://paxton.com",
], "semantic_type": "",
"description": ""}],
"column":
"user ID", "properties": {"dtype": "number",
"std": 29, "min": 1, "max": 100,
"num_unique_values": 74, "samples": [6, 87, 23],
"semantic_type": "",
"description": ""}],
"column":
"created dat", "properties": {"dtype":
"date", "min": "2023-04-13 08:04:00", "max":
"2023-04-13 08:04:00", "num_unique_values": 1,
"samples": ["2023-04-13 08:04:00",
], "semantic_type": "",
"description": ""}]}
```

```

n    },\n    {\n        \"column\": \"Insta filter used\", \n        \"properties\": {\n            \"dtype\": \"category\", \n            \"num_unique_values\": 2, \n            \"samples\": [\n                \"no\" \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n        }, \n        {\n            \"column\": \"photo type\", \n            \"properties\": {\n                \"dtype\": \"category\", \n                \"num_unique_values\": 3, \n                \"samples\": [\n                    \"photo\" \n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\" \n            }, \n            {\n                \"column\": \n                \"Posting_Hour\", \n                \"properties\": {\n                    \"dtype\": \n                    \"int32\", \n                    \"num_unique_values\": 1, \n                    \"samples\": [\n                        8 \n                    ], \n                    \"semantic_type\": \"\", \n                    \"description\": \"\" \n                }, \n                {\n                    \"column\": \n                    \"Posting_Day\", \n                    \"properties\": {\n                        \"dtype\": \n                        \"category\", \n                        \"num_unique_values\": 1, \n                        \"samples\": \n                        [\n                            \"Thursday\" \n                        ], \n                        \"semantic_type\": \n                        \"\", \n                        \"description\": \"\" \n                    }, \n                    {\n                        \"column\": \"Posting_Month\", \n                        \"properties\": {\n                            \"dtype\": \"category\", \n                            \"num_unique_values\": 1, \n                            \"samples\": [\n                                \"April\" \n                            ], \n                            \"semantic_type\": \"\", \n                            \"description\": \"\" \n                        } \n                    } \n                } \n            ], \n        }, \n    ], \n    \"type\": \"dataframe\", \n    \"variable_name\": \"photos\" \n}

```

```

print(likes.columns)
print(comments.columns)
print(tags.columns)
print(photo_tags.columns)
print(follows.columns)

```

```

Index(['user ', 'photo', 'created time', 'following or not', 'like type'], dtype='object')
Index(['id', 'comment', 'User id', 'Photo id', 'created Timestamp', 'posted date', 'emoji used', 'Hashtags used count'], dtype='object')
Index(['id', 'tag text', 'created time', 'location'], dtype='object')
Index(['photo', 'tag ID', 'user id'], dtype='object')
Index(['follower', 'followee ', 'created time', 'is follower active', 'followee Acc status'], dtype='object')

```

```

print(users.columns)
print(photos.columns)

```

```

Index(['id', 'name', 'created time', 'private/public', 'post count', 'Verified status'], dtype='object')
Index(['id', 'image link', 'user ID', 'created dat', 'Insta filter used', 'photo type'], dtype='object')

```

```

# -----
# Calculate follower count
# -----

follower_count = (
    follows.groupby('followee ')
    .size()
    .reset_index(name='Followers')
)

follower_count.rename(columns={'followee ': 'user ID'}, inplace=True)

# -----
# Calculate likes per photo
# -----

likes_count = (
    likes.groupby('photo')
    .size()
    .reset_index(name='Likes')
)

likes_count.rename(columns={'photo': 'id'}, inplace=True)

# -----
# Calculate comments per photo
# -----

comments_count = (
    comments.groupby('Photo id')
    .size()
    .reset_index(name='Comments')
)

comments_count.rename(columns={'Photo id': 'id'}, inplace=True)

# -----
# Merge with photos
# -----

analysis = photos.merge(likes_count, on='id', how='left')
analysis = analysis.merge(comments_count, on='id', how='left')

analysis['Likes'] = analysis['Likes'].fillna(0)
analysis['Comments'] = analysis['Comments'].fillna(0)

# -----
# Merge follower counts

```

```

# -----

analysis = analysis.merge(follower_count, on='user ID', how='left')

analysis['Followers'] = analysis['Followers'].fillna(1)

# -----
# Engagement Rate
# -----

analysis['Engagement Rate'] = (
    (analysis['Likes'] + analysis['Comments'])
    / analysis['Followers']
) * 100

analysis.head()

{"summary":{"\n  \"name\": \"analysis\",\n  \"rows\": 257,\n  \"fields\": [\n    {\n      \"column\": \"id\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 74,\n        \"min\": 1,\n        \"max\": 257,\n        \"num_unique_values\": 257,\n        \"samples\": [\n          166,\n          187,\n          83\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\",\n        \"image link\": \"\",\n        \"properties\": {\n          \"dtype\": \"string\",\n          \"num_unique_values\": 257,\n          \"samples\": [\n            \"http://kaleigh.biz\",\n            \"http://germaine.name\",\n            \"http://paxton.com\"\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          \"user ID\": \"\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 29,\n            \"min\": 1,\n            \"max\": 100,\n            \"num_unique_values\": 74,\n            \"samples\": [\n              6,\n              87,\n              23\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\",\n            \"created date\": \"\",\n            \"properties\": {\n              \"dtype\": \"object\",\n              \"num_unique_values\": 1,\n              \"samples\": [\n                \"13-04-2023 08:04\"\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\": \"Insta filter used\",\n              \"properties\": {\n                \"dtype\": \"category\",\n                \"num_unique_values\": 2,\n                \"samples\": [\n                  \"no\"\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n              },\n              {\n                \"column\": \"photo type\",\n                \"properties\": {\n                  \"dtype\": \"category\",\n                  \"num_unique_values\": 3,\n                  \"samples\": [\n                    \"photo\"\n                  ],\n                  \"semantic_type\": \"\",\n                  \"description\": \"\"\n                },\n                {\n                  \"column\": \"Likes\",\n                  \"properties\": {\n                    \"dtype\": \"number\",\n                    \"std\": 3,\n                    \"min\": 25,\n                    \"max\": 48,\n
```

```

{"num_unique_values": 20, "samples": [25], "semantic_type": "", "description": ""}
{"column": "Comments", "properties": {"dtype": "number", "std": 3, "min": 21, "max": 39, "num_unique_values": 19, "samples": [25], "semantic_type": "", "description": ""}
{"column": "Followers", "properties": {"dtype": "number", "std": 0, "min": 76, "max": 77, "num_unique_values": 2, "samples": [76], "semantic_type": "", "description": ""}
{"column": "Engagement Rate", "properties": {"dtype": "number", "std": 6.360015380273168, "min": 64.93506493506493, "max": 103.94736842105263, "num_unique_values": 46, "samples": [93.5064935064935], "semantic_type": "", "description": ""}
}, "type": "dataframe", "variable_name": "analysis"}

plt.figure(figsize=(10,6))

bars = plt.bar(posting_day.index, posting_day.values)

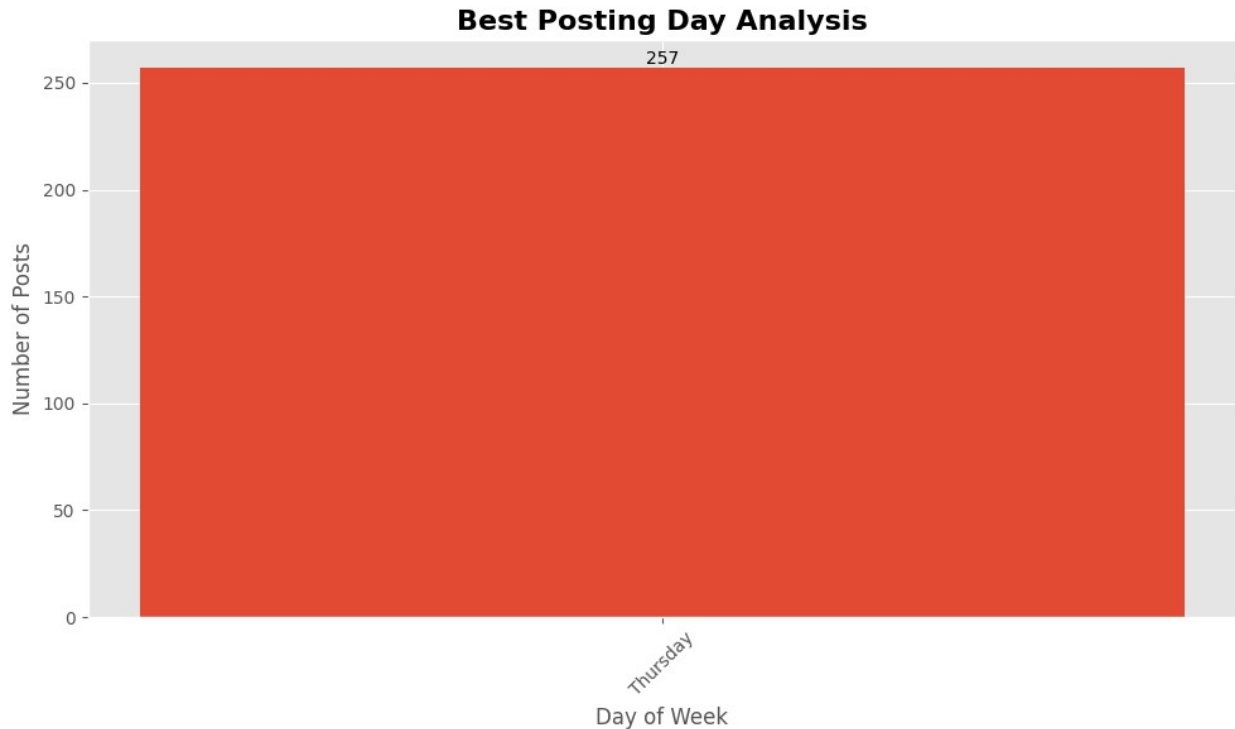
plt.title("Best Posting Day Analysis", fontsize=16, fontweight="bold")
plt.xlabel("Day of Week")
plt.ylabel("Number of Posts")

for bar in bars:
    height = bar.get_height()

    if not pd.isna(height): # Ignore NaN values
        plt.text(
            bar.get_x() + bar.get_width()/2,
            height,
            f"{int(height)}",
            ha='center',
            va='bottom'
        )

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



```
# Merge photo_tags with tags
hashtags = photo_tags.merge(
    tags,
    left_on='tag ID',
    right_on='id',
    how='left'
)

# Count hashtag usage
top_hashtags = (
    hashtags['tag text']
    .value_counts()
    .head(10)
)

# Plot
plt.figure(figsize=(12,6))

bars = plt.bar(top_hashtags.index, top_hashtags.values)

plt.title("Top 10 Most Used Hashtags", fontsize=16, fontweight='bold')
plt.xlabel("Hashtags")
plt.ylabel("Frequency")

plt.xticks(rotation=45, ha='right')

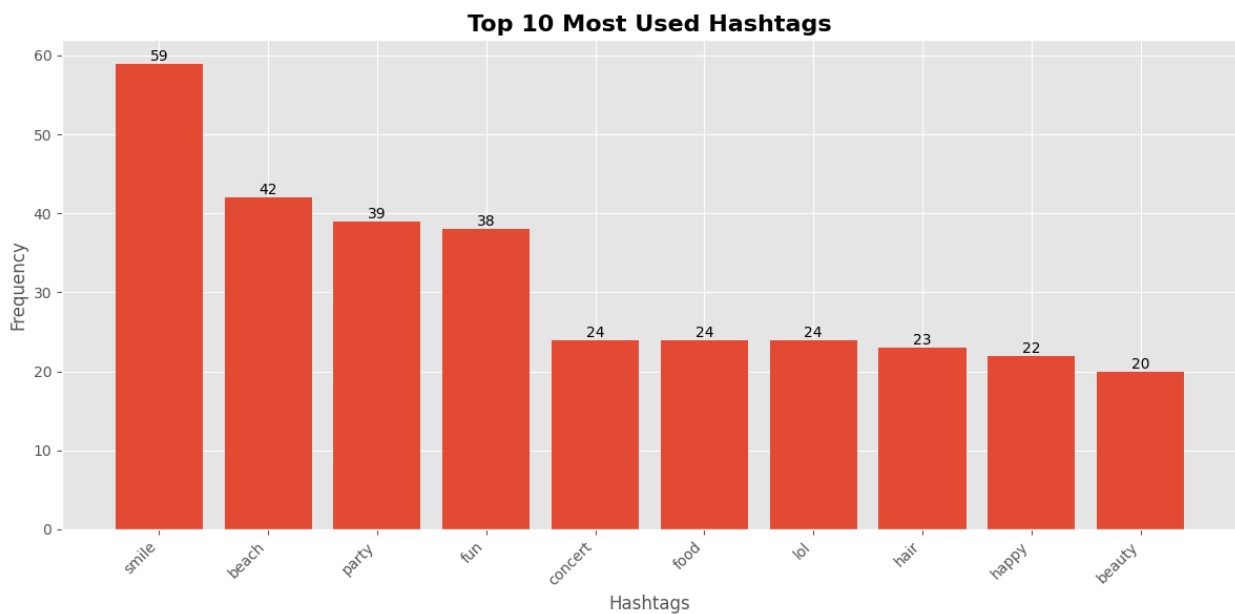
# Add labels on bars
```

```

for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width()/2,
        height,
        str(int(height)),
        ha='center',
        va='bottom'
    )

plt.tight_layout()
plt.show()

```



```

import matplotlib.pyplot as plt

# Recommendations
recommendations = [
    "Use Popular Hashtags",
    "Post Consistently",
    "Use Trending Filters",
    "Increase Public Posts",
    "Encourage User Engagement"
]

# Importance scores (for visualization)
scores = [10, 9, 8, 8, 9]

# Plot
plt.figure(figsize=(10,6))

```

```

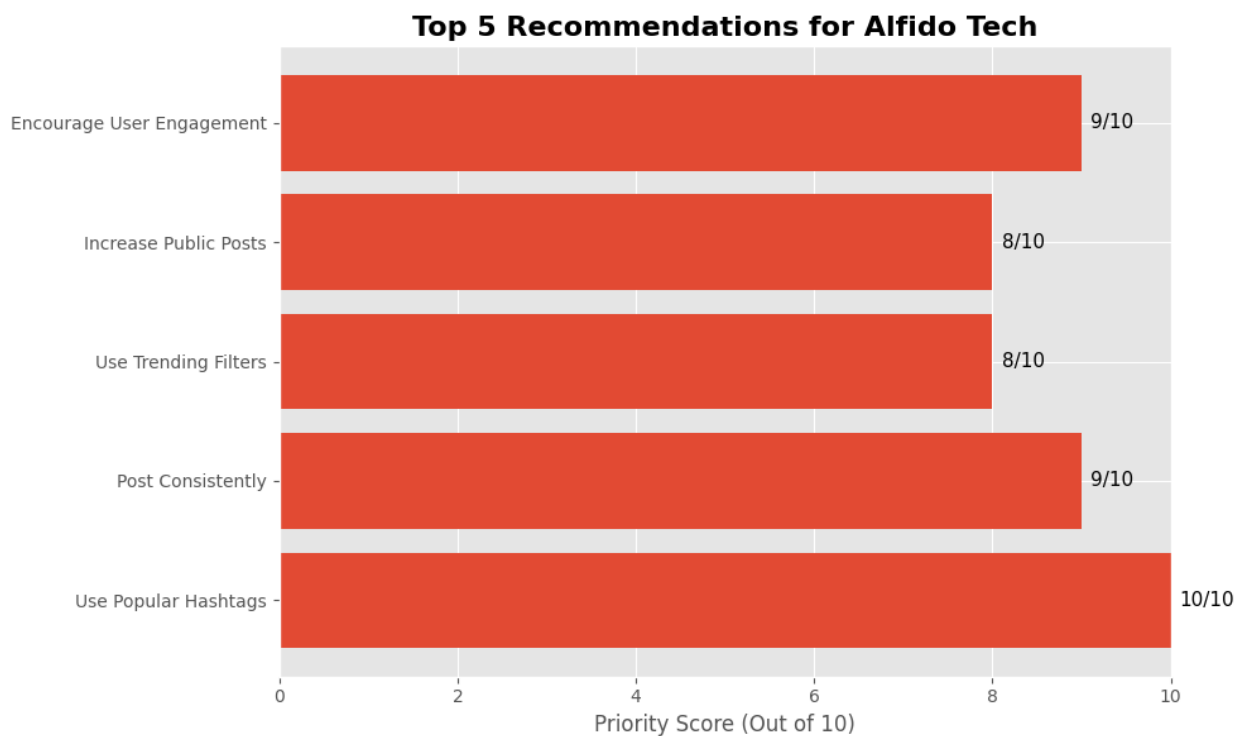
bars = plt.barh(recommendations, scores)

plt.title("Top 5 Recommendations for Alfido Tech", fontsize=16,
fontweight='bold')
plt.xlabel("Priority Score (Out of 10)")
plt.xlim(0, 10)

# Add score labels
for bar in bars:
    width = bar.get_width()
    plt.text(width + 0.1,
             bar.get_y() + bar.get_height()/2,
             f"{width}/10",
             va='center',
             fontsize=11)

plt.tight_layout()
plt.show()

```



```

import matplotlib.pyplot as plt

# Weekly Content Calendar
days = [
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",

```

```

    "Friday",
    "Saturday",
    "Sunday"
]

# Recommended number of posts
posts = [1, 1, 1, 1, 1, 2, 1]

plt.figure(figsize=(10,6))

bars = plt.bar(days, posts)

plt.title("Recommended Weekly Content Calendar for Alfido Tech",
          fontsize=16,
          fontweight='bold')

plt.xlabel("Day of Week")
plt.ylabel("Recommended Posts")

# Add labels
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width()/2,
        height + 0.03,
        str(int(height)),
        ha='center',
        fontsize=11
    )

plt.ylim(0,3)
plt.tight_layout()
plt.show()

```

